

MarketsGrid Usage Guide

MarketsGrid Usage Guide

Audience: application developers integrating `@starui/grid` into browser or OpenFin apps.

Scope: how to choose and wire the three React entry points (`MarketsGrid` , `MarketsGridContainer` , `HostedMarketsGrid`), bootstrap the SharedWorker data hub, attach providers, and persist grid state — across common deployment scenarios.

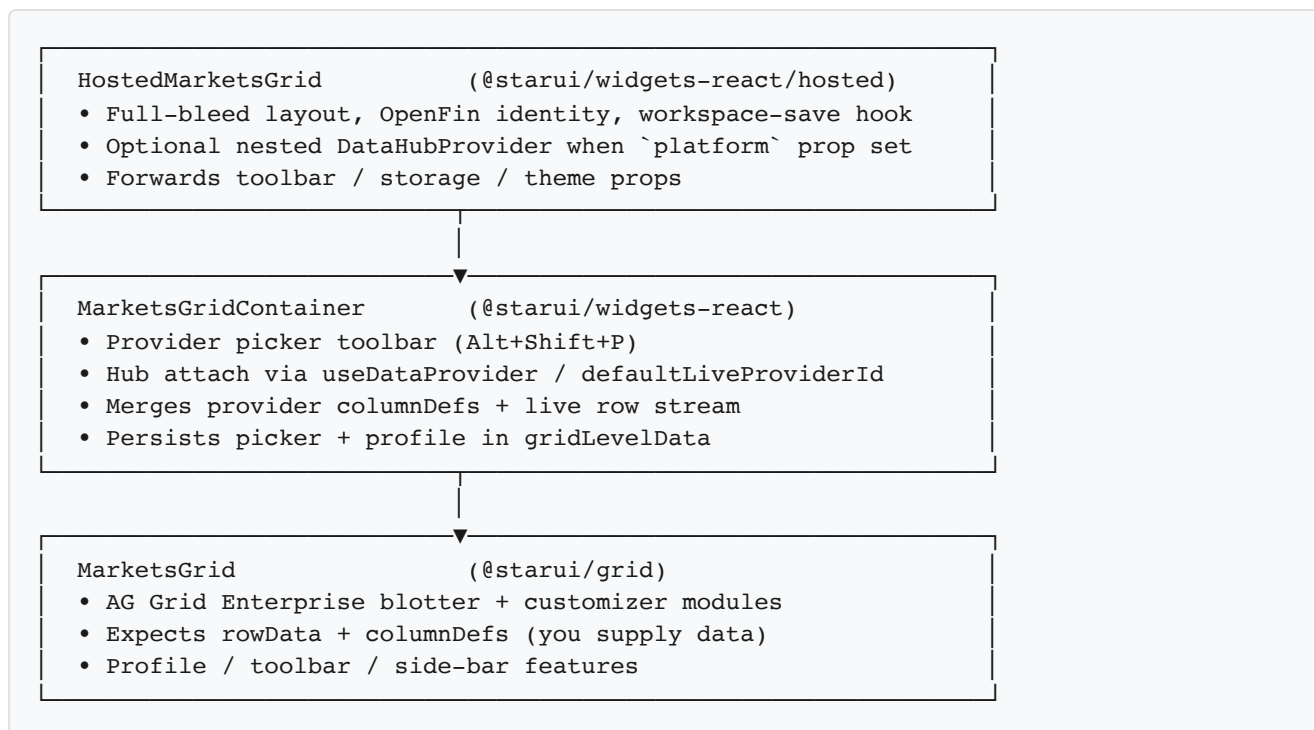
Related docs:

Document	Focus
<code>STOMP_DATAPROVIDER_MARKETSGRID_GUIDE.md</code>	Step-by-step STOMP wiring
<code>guides/platform-bootstrap-config.md</code>	<code>appId</code> / <code>userId</code> / REST bootstrap
<code>guides/consumer-app-sharedworker-and-tailwind.md</code>	Vite + SharedWorker consumer setup
<code>PROFILE_PERSISTENCE.md</code>	Profile keys, workspace save, storage adapters
<code>ARCHITECTURE.md</code>	Monorepo layer model

PDF: `MARKETSGRID_USAGE_GUIDE.pdf` — regenerate with `npm run docs:marketsgrid-usage-pdf` from the repo root.

1. Component model — three layers

MarketsGrid is never “just drop in a grid” in production streaming apps. Pick the layer that matches how much wiring you want to own.



When to use which

Layer	Use when	You supply	Hub required?
MarketsGrid	Feature lab, static/mock rows, custom data pipeline	rowData , columnDefs , optional storage	No
MarketsGridContainer	Custom page layout; you own chrome but want provider picker + hub attach	gridId , optional defaultLiveProviderId	Yes (DataHubProvider ancestor)
HostedMarketsGrid	Production blotter in browser or OpenFin view	componentName , defaultInstanceId , provider id or picker	Yes (ancestor or platform prop)

Rule of thumb: streaming market data → **HostedMarketsGrid** (simplest) or **MarketsGridContainer** (embedded in your layout). Static demos → **MarketsGrid** only.

2. Data hub prerequisite (streaming scenarios)

All provider-driven grids share the same runtime backbone:

```

App boot
└─ ensurePlatformReady(config, { workerScriptUrl })
    └─ Main-thread ConfigManager.init()      → IndexedDB (Dexie)
    └─ SharedWorker spawn
        └─ Worker ConfigManager.init()
            └─ hydrateCatalog()              → ConfigCatalogCache
            └─ hydrateAppData()              → WorkerAppDataStore
    └─ wait: AppData mirror + catalog ready

React
└─ DataHubProvider(platform)
    └─ client      → MessagePort RPC
    └─ appData     → AppData mirror ({{name.key}} templates)
    └─ configStore → save/list provider rows (main thread)

Grid attach
└─ attach(providerId)  cfg-free if id is in worker catalog
    └─ lazy ProviderSlot + STOMP/mock/rest upstream + row cache

```

Identity (`appId` , `userId`) is **deployment-wide** — from `public/app-config.json` (browser) or OpenFin manifest `customSettings` . Per-grid `instanceId` / `gridId` are separate (profiles, picker state).

3. Scenario catalog

#	Scenario	Entry component	Hub	Example app
A	Minimal browser STOMP blotter	<code>HostedMarketsGrid</code>	Yes	<code>apps/stomp-marketsgrid-minimal</code>
B	Browser STOMP + OpenFin option	<code>HostedMarketsGrid</code>	Yes	<code>apps/demo-stomp-markets-grid</code>
C	Provider editor + dual grids	<code>HostedMarketsGrid</code> ×2	Yes	<code>apps/tutorials-workspace/dataprovider-editor</code>
D	STOMP tutorial (workspace tarballs)	<code>HostedMarketsGrid</code>	Yes	<code>apps/tutorials-workspace/stomp</code>
E	OpenFin workspace blotter	<code>HostedMarketsGrid</code>	Yes	<code>apps/e2e/openfin-workspace</code>
F	OpenFin production reference	<code>HostedMarketsGrid</code>	Yes	<code>apps/legacy/markets-ui-react-reference</code>
G	E2E browser modes	<code>HostedMarketsGrid</code> / standalone	Optional	<code>apps/e2e/browser-blotter</code>
H	Grid feature lab (static rows)	<code>MarketsGrid</code>	No	<code>apps/markets-grid-lab</code>

#	Scenario	Entry component	Hub	Example app
H2	Basic tutorial (localStorage)	MarketsGrid	No	apps/tutorials-workspace/basic
I	Mock provider + hub	HostedMarketsGrid	Yes	apps/tutorials-workspace/mockdata-provider
J	REST config service	HostedMarketsGrid	Yes	apps/legacy/demo-configservice-react

4. Scenario A — Minimal browser STOMP blotter

Goal: smallest path from zero to live STOMP rows. No provider editor UI.

Reference: apps/stomp-marketsgrid-minimal

Boot sequence

1. bootstrap() → resolvePlatformBootstrapFromJson + ensurePlatformReady
2. DataHubProvider(platform) wraps the tree
3. App seeds catalog row via configStore.save(stompProviderDraft) (idempotent)
4. HostedMarketsGrid with defaultLiveProviderId={providerId} — **cfg-free attach**

Minimal code shape

```
// main.tsx — boot BEFORE render
void bootstrap().then(({ config, platform }) => {
  root.render(
    <DataHubProvider platform={platform} userId={config.userId}>
      <App />
    </DataHubProvider>,
  );
});

// App.tsx
<HostedMarketsGrid
  gridId="stomp-blotter"
  componentName="STOMP Positions"
  defaultInstanceId="stomp-blotter"
  defaultLiveProviderId={providerId}
  withStorage
  configManager={getPlatform().configManager}
/>
```

Prerequisites

- STOMP broker: npm run dev:stomp (ws://localhost:8081)
- public/app-config.json with stable appId / userId
- Vite consumer config with { worker: true }

Dev tooling

- **Alt+Shift+S** — hub inspector (providers, subscribers, cache sizes, cfg JSON)
 - **Alt+Shift+P** — provider toolbar (hidden by default in minimal app; grid still auto-attaches via `defaultLiveProviderId`)
-

5. Scenario B — Browser STOMP demo with chrome

Goal: same hub path as A, with optional OpenFin launch and richer demo shell.

Reference: `apps/demo-stomp-markets-grid`

Same bootstrap + `HostedMarketsGrid` pattern. May include tabs, help copy, and `npm run openfin` for platform manifest testing.

6. Scenario C — Provider editor + multiple grids

Goal: author providers in UI; run two independent grids on one SharedWorker hub.

Reference: `apps/tutorials-workspace/dataprovider-editor`

Layout pattern

```
DataHubProvider (one hub per appId)
├── Dock layout
│   ├── HostedGridPanel A    gridId="grid-a"
│   └── HostedGridPanel B    gridId="grid-b"
└── DataProviderEditor        configStore.save → invalidate worker catalog
```

Key behaviors

- **One hub, many subscribers:** each grid `attach(providerId)` adds a data listener; upstream STOMP/mock connection is **shared** per `providerId`.
- **Separate profiles:** each grid has its own `gridId` / `instanceId` → separate `gridLevelData` (picker selection + MarketsGrid profile).
- **Picker toolbar:** Alt+Shift+P reveals live/historical provider selection per grid.

Embedded panel note

When `HostedMarketsGrid` sits inside a dock panel (not viewport root), wrap it in a positioned container so its internal `position: fixed` full-bleed layout pins to the **panel**, not the window. See `HostedGridPanel.tsx` in the tutorial app.

7. Scenario D — STOMP tutorial app

Goal: guided STOMP setup with help sheet and seeded provider utilities.

Reference: `apps/tutorials-workspace/stomp` , `apps/tutorials-tarball/stomp`

Uses `ensurePlatformReady` + `DataHubProvider` + `PositionsBlotter.tsx` rendering `HostedMarketsGrid` . Good middle ground between minimal and dataprovider-editor complexity.

8. Scenario E — OpenFin workspace view

Goal: MarketsGrid as an OpenFin **view** inside a workspace platform.

Reference: `apps/e2e/openfin-workspace`

Differences from plain browser

Concern	Browser	OpenFin
Bootstrap config	<code>public/app-config.json</code>	Manifest <code>customSettings.appId/userId</code>
<code>instanceId</code>	<code>defaultInstanceId</code> prop	View <code>customData.instanceId</code> (fallback to default)
ConfigManager	Explicit from <code>getPlatform()</code>	Often OpenFin singleton; pass override in tests
Workspace save	N/A	<code>HostedMarketsGrid</code> registers <code>workspace-saving</code> → flush grid profile
Tab strip / caption	N/A	<code>tabsHidden</code> + <code>caption</code> when platform hides tabs

Wiring (same hub pattern)

```
void initPlatformBootstrap().then(({ config, platform }) => {
  root.render(
    <DataHubProvider platform={platform} userId={config.userId}>
      <Blotter />    {/* HostedMarketsGrid inside */}
    </DataHubProvider>,
  );
});
```

Manifest view URL typically includes `?view=blotter` . Platform provider view spawns with `customData.instanceId` per blotter instance.

9. Scenario F — Production OpenFin reference

Goal: full platform shell (dock, registry, config browser routes).

Reference: `apps/legacy/markets-ui-react-reference`

Multiple routes render `HostedMarketsGrid` via thin view wrappers (`BlottersMarketsGrid.tsx`). Uses `ensurePlatformReady` + `DataHubProvider` at app root. Study this for multi-blotter production layouts.

10. Scenario G — E2E browser blotter modes

Reference: `apps/e2e/browser-blotter`

Mode	Grid wiring	Purpose
<code>standalone</code>	In-app rows, no hub	Baseline UI without SharedWorker
<code>provider / config / full</code>	<code>HostedMarketsGrid</code> + hub	Integration / toolbar / config flows

Useful when testing attach semantics (`data-status="wired"`) without OpenFin.

11. Scenario H — Feature lab (no hub)

Goal: exercise MarketsGrid modules (formatting, alerts, profiles, editing) with **static** `rowData` .

Reference: `apps/markets-grid-lab`

```
import { MarketsGrid } from '@starui/grid';

<MarketsGrid
  gridId="lab-formatting"
  rowData={rows}
  columnDefs={cols}
  showFormattingToolbar
  storage={localStorageAdapter}
/>
```

No `DataHubProvider` , **no** `ensurePlatformReady` , **no** `SharedWorker`. Data never flows through the hub. Use for UI/feature development only — not for STOMP integration testing.

12. Scenario I — Mock provider + hub

Goal: synthetic streaming data without external broker.

Reference: `apps/tutorials-workspace/mockdata-provider`

Same hub bootstrap as STOMP scenarios. Provider `providerType: 'mock'` in catalog. `HostedMarketsGrid` OR `MarketsGridContainer` attaches by id. Ideal for CI, demos offline, and e2e `openfin-workspace` mock provider.

13. Scenario J — REST config service

Goal: ConfigManager talks to remote REST API (Dexie as cache + pending sync queue).

Reference: `apps/legacy/demo-configservice-react`

Set in bootstrap config:

```
{
  "appId": "my-app",
  "userId": "dev1",
  "useRest": true,
  "configServiceRestUrl": "http://localhost:3001/api/v1"
}
```

Grids still use `DataHubProvider` + `HostedMarketsGrid`. Provider rows persist via REST; worker catalog hydrates from the same ConfigManager API on the worker side. Config Browser reads local Dexie cache on main thread.

14. Provider attachment modes

Recommended: cfg-free attach (catalog)

1. Save provider via `configStore.save()` (editor or programmatic seed).
2. Worker catalog reloads via `client.invalidateConfig()`.
3. Grid passes `defaultLiveProviderId` or user picks in toolbar.
4. Hub resolves transport cfg from `ConfigCatalogCache` — **no inline cfg on attach**.

Legacy: inline cfg on attach

Pass full `ProviderConfig` on first attach when id is not in catalog. Deprecated for saved providers — use catalog + cfg-free attach.

AppData templates

When cfg contains `{{positions.asOfDate}}`:

1. Values live in AppData rows (worker-persisted).
 2. Main thread resolves via `useResolvedCfg` before attach.
 3. Use `DataHubProvider mode="eager"` if first attach must wait for AppData snapshot.
-

15. Persistence & identity keys

Key	Source	Stored in	Purpose
<code>appId</code>	Bootstrap config	SharedWorker name	One hub per deployment
<code>userId</code>	Bootstrap config	Provider visibility, AppData ownership	Session user

Key	Source	Stored in	Purpose
<code>gridId</code>	Prop on grid	Profile namespace	Column layout, filters, modules
<code>instanceId</code>	OpenFin customData or <code>defaultInstanceId</code>	Storage adapter scope	Profile bundle key with <code>appId/userId</code>
<code>providerId</code>	ConfigManager row id	<code>appConfig</code> table	Hub catalog + attach target

With `withStorage={true}`, `HostedMarketsGrid` builds a ConfigService-backed `StorageAdapterFactory` from `configManager`.

See [PROFILE_PERSISTENCE.md](#) for workspace-save timing and OpenFin Channel wiring.

16. Bootstrap placement patterns

Pattern 1 — External boot (recommended for clarity)

```
const { platform, config } = await bootstrap(); // ensurePlatformReady
<DataHubProvider platform={platform} userId={config.userId}>
  <App />
</DataHubProvider>
```

Used by: `stomp-marketsgrid-minimal`, `openfin-workspace`, `markets-grid-lab` (hub tabs only).

Pattern 2 — Self-bootstrapping provider

```
<DataHubProvider bootstrapConfig={config} workerScriptUrl={workerAssetUrl}>
  <App />
</DataHubProvider>
```

`ensurePlatformReady` runs inside the provider. Equivalent outcome; pick one style per app.

Pattern 3 — Platform on HostedMarketsGrid

```
<HostedMarketsGrid platform={platform} ... />
```

Mounts a **nested** `DataHubProvider`. Avoid double-wrapping if ancestor already provides hub context.

17. OpenFin vs browser checklist

Browser app checklist

- ☐ `public/app-config.json` with `appId`, `userId`
- ☐ `vite.config` → `staruiConsumerViteConfig(..., { worker: true })`
- ☐ Worker asset: `@starui/host-data/assets/data-services-worker.mjs?url`

- ☐ `ensurePlatformReady` before render
- ☐ `DataHubProvider` wrapping grid tree
- ☐ Provider row in catalog (save or editor)
- ☐ `HostedMarketsGrid` with `defaultLiveProviderId` or picker
- ☐ STOMP broker running (if using STOMP)

OpenFin app checklist

- ☐ Manifest `customSettings` matches bootstrap shape
- ☐ Platform provider initializes workspace
- ☐ View manifest URL + `customData.instanceId`
- ☐ Same hub bootstrap in view entry (`initPlatformBootstrap`)
- ☐ `HostedMarketsGrid` with `withStorage` + `configManager`
- ☐ Test workspace save flushes grid state

18. Troubleshooting

Symptom	Likely cause	Action
Blank screen then grid	Normal while seeding <code>providerId</code>	Wait; check console
Grid stuck <code>loading</code>	Broker down / wrong WebSocket URL	<code>npm run dev:stomp</code> ; verify <code>websocketUrl</code>
Empty grid, status <code>ready</code>	Wrong <code>keyColumn</code> vs row shape	Match provider <code>keyColumn</code> to STOMP JSON
Provider not found on attach	Catalog not invalidated after save	Ensure <code>configStore.save</code> completed; check hub inspector
Two tabs, stale provider list	Same <code>appId</code> — shared hub is correct	Expected; both share one upstream per <code>providerId</code>
Picker empty	No saved providers for user/subtype	Save via editor or programmatic seed
Profiles not saving	<code>withStorage</code> false or no <code>configManager</code>	Pass both props
OpenFin view wrong profile	<code>instanceId</code> collision	Unique <code>customData.instanceId</code> per view

Hub inspector (dev): Alt+Shift+S on any `DataHubProvider` app in development.

19. Package imports cheat sheet

```
// Grid primitive (static data)
import { MarketsGrid } from '@starui/grid';

// Provider-aware container
import { MarketsGridContainer } from '@starui/widgets-react';

// Production hosted shell (browser + OpenFin)
import { HostedMarketsGrid } from '@starui/widgets-react/hosted';

// Hub bootstrap
import {
  ensurePlatformReady,
  resolvePlatformBootstrapFromJson,
} from '@starui/host-data';
import workerAssetUrl from '@starui/host-data/assets/data-services-worker.mjs?url';

// React hub context + hooks
import {
  DataHubProvider,
  useDataServices,
  useDataProvider,
  useUserIdFromContext,
} from '@starui/host-data-react/runtime';
```

20. Choosing your starting template

You want...	Start here
Absolute minimum STOMP grid	apps/stomp-marketsgrid-minimal
STOMP + narrative / OpenFin launch	apps/demo-stomp-markets-grid
Learn provider editor + dual grids	apps/tutorials-workspace/dataprovider-editor
OpenFin view integration test	apps/e2e/openfin-workspace
Full OpenFin platform reference	apps/legacy/markets-ui-react-reference
Grid UI features without hub	apps/markets-grid-lab
MCP scaffold from scratch	@starui/mcp-scaffold templates stomp , openfin-platform , dataprovider-editor

Document history

Date	Change
2026-05-28	Initial comprehensive scenario guide